

```
for i := 0; i < data.Rows; i++ {
    if !data.RowHasMissingValues(i) {
        filteredData.AppendRow(data.Row(i))
    }
}
```

Explanation:

- *RowHasMissingValues(i)* checks if a row contains missing values.
- *AppendRow(data.Row(i))* appends only valid rows to the new dataset.

Feature scaling and normalisation

```
import "github.com/sjwhitworth/golearn/filters"

// Normalize data using Min-Max scaling
normalizer := filters.NewMinMaxScaler("FeatureColumn")
normalizer.Fit(data) // Compute min-max scaling parameters
normalizedData := normalizer.Transform(data) // Apply transformation
```

Explanation:

- *NewMinMaxScaler("FeatureColumn")* creates a Min-Max scaler for a given feature.
- *Fit(data)* computes scaling parameters from the dataset.
- *Transform(data)* applies normalisation.

Encoding categorical variables

GoLearn requires categorical values to be encoded numerically.

```
import "github.com/sjwhitworth/golearn/base"

// Convert categorical labels to numerical form
data.ConvertToCategorical(0) // Convert the first column to categorical values
```

Explanation:

- *ConvertToCategorical(0)* converts the first column into numeric values, required for ML models.

Implementing a classification model**Example: Building a decision tree classifier**

```
import (
    "github.com/sjwhitworth/golearn/trees"
)

// Create a decision tree classifier
tree := trees.NewID3DecisionTree(0.6) // Decision threshold:
```

```
0.6
tree.Fit(trainingData) // Train the model

// Make predictions
predictions, _ := tree.Predict(testData)
```

Explanation:

- *NewID3DecisionTree(0.6)* initialises a decision tree with a confidence threshold.
- *Fit(trainingData)* trains the model.
- *Predict(testData)* generates predictions.

Training the model and evaluating performance

```
import "github.com/sjwhitworth/golearn/evaluation"

// Compute accuracy
accuracy, _ := evaluation.GetAccuracy(predictions, testData)
fmt.Println("Model Accuracy:", accuracy)
```

Explanation:

- *GetAccuracy(predictions, testData)* computes the model's accuracy.

Regression with GoLearn**Example: Implementing linear regression**

```
import (
    "github.com/sjwhitworth/golearn/linear_models"
)

// Create a linear regression model
linReg := linear_models.NewLinearRegression()
linReg.Fit(trainingData) // Train model

// Make predictions
predictions, _ := linReg.Predict(testData)
```

Explanation:

- *NewLinearRegression()* initialises a linear regression model.
- *Fit(trainingData)* trains the model.
- *Predict(testData)* generates predictions.

Visualising and interpreting results

GoLearn does not provide built-in visualisation tools, but you can export results and use Python/Matplotlib for plotting.

```
import "os"

// Save predictions to a CSV file
file, _ := os.Create("predictions.csv")
defer file.Close()
```